Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3712256.3726372

RESEARCH-ARTICLE

Learning Adaptive Neighborhood Search with Dual Operator Selection for Capacitated Vehicle Routing Problem

XIANG-LING CHEN, Victoria University of Wellington, Wellington, WGN, New Zealand

YI MEI, Victoria University of Wellington, Wellington, WGN, New Zealand

MENGJIE ZHANG, Victoria University of Wellington, Wellington, WGN, New Zealand

Open Access Support provided by:

Victoria University of Wellington

PDF Download
3712256.3726372.pdf
21 March 2026
Total Citations: 0
Total Downloads: 917

Published: 13 July 2025

Citation in BibTeX format

GECCO '25: Genetic and Evolutionary Computation Conference
July 14 - 18, 2025
Malaga, Spain

Conference Sponsors:
SIGEVO

GECCO '25: Proceedings of the Genetic and Evolutionary Computation Conference (July 2025)
https://doi.org/10.1145/3712256.3726372
ISBN: 9798400714658

Learning Adaptive Neighborhood Search with Dual Operator Selection for Capacitated Vehicle Routing Problem

Xiang-Ling Chen, Yi Mei, Mengjie Zhang
Centre for Data Science and Artificial Intelligence & School of Engineering and Computer Science
Victoria University of Wellington, New Zealand
{xiangling.chen,yi.mei,mengjie.zhang}@ecs.vuw.ac.nz

ABSTRACT

The Capacitated Vehicle Routing Problem (CVRP) is a classic optimization problem with widespread applications in real-world scenarios. Existing research has developed numerous neighborhood operators capable of generating high-quality solutions. However, most neighborhood search algorithms either apply all operators in a predefined/random sequence or adaptively adjust only improvement operators, neglecting the learning of perturbation operators. This limitation often results in suboptimal performance. To address this issue, this paper proposes Learning Adaptive Neighborhood Search with Dual operator Selection (LANDS), an algorithm that integrates two RL-based controllers to adaptively select both improvement and perturbation operators simultaneously. Within this framework, the two controllers act as high-level guides, collaboratively directing the optimization process by selecting appropriate operators. Additionally, to reduce resource waste, a filtering mechanism is introduced to exclude operators deemed ineffective within the same improvement iteration, enhancing the algorithm's efficiency. The effectiveness of the proposed LANDS method is verified by a series of experiments.

CCS CONCEPTS

• **Mathematics of computing → Combinatorial optimization**; • **Computing methodologies → Optimization algorithms**; *Discrete space search.*

KEYWORDS

adaptive operator selection, learn to optimize, meta-heuristic, capacitated vehicle routing problem

ACM Reference Format:
Xiang-Ling Chen, Yi Mei, Mengjie Zhang. 2025. Learning Adaptive Neighborhood Search with Dual Operator Selection for Capacitated Vehicle Routing Problem. In *Genetic and Evolutionary Computation Conference (GECCO '25), July 14–18, 2025, Málaga, Spain.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3712256.3726372

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
GECCO '25, July 14–18, 2025, Málaga, Spain
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1465-8/2025/07
https://doi.org/10.1145/3712256.3726372

1 INTRODUCTION

The Vehicle Routing Problem (VRP) is a fundamental and widely studied optimization problem with numerous practical applications in various industries, such as logistics, transportation, and delivery services. Its objective is to determine efficient routes for a fleet of vehicles to service a set of customers while minimizing the total costs, typically measured in terms of travel distances or time. One of the most important variants of the VRP is the Capacitated Vehicle Routing Problem (CVRP), which introduces a vehicle capacity constraint. In CVRP, the goal is to find a set of routes that meet customer demands without exceeding vehicle capacity. Given their relevance to real-world problems, the solution of VRP is of significant importance to improve operational efficiency, reduce costs, and improve service quality in industries such as delivery services and transportation. As the demand for large-scale, real-time routing solutions continues to grow, the need for effective algorithms capable of handling complex environments becomes increasingly critical [1, 2].

Solving the VRP is a challenging task due to its NP-hardness [3]. The exact algorithms [4–6], which guarantee optimal solutions, have been widely studied for VRP. However, their high time complexity makes them computationally infeasible for large-scale instances, and they are typically only applicable to small-scale VRP problems. On the other hand, heuristic [7–9] and meta-heuristic [10–13] methods have become a popular alternative, as they can provide satisfactory near-optimal solutions within reasonable time limits. Among these methods, a significant amount of effort has been dedicated to designing neighborhood search algorithms [14, 15] that utilize various neighborhood search operators (such as local search [16], mutation [17] and crossover [18, 19]), which have shown promising results. In neighborhood search algorithms, operators can be categorized into two types: improvement operators, which aim to refine the current solution, and perturbation operators, which introduce diversity into the search space to jump out of local optima by making larger changes to the solution. The core idea behind the neighborhood search algorithm is to iteratively perturb or improve a solution within a given local neighborhood. In this context, the correct selection of the appropriate operators for the current search state becomes crucial to obtaining high-quality solutions. The diverse characteristics of operators make it challenging to determine in advance which ones will be most effective for a given problem. Thus, ensuring that the suitable operators are chosen [20] at each step is essential for the success of these algorithms, as improper or random choices can lead to suboptimal performance and wasted computational resources.

To address this challenge, Adaptive Operator Selection (AOS) techniques [21–24] have been proposed, aiming to improve the performance of heuristic search algorithms by dynamically selecting the most appropriate operators during the optimization process. Unlike traditional methods, which apply operators in a fixed or random sequence, AOS leverages feedback from the search process to adaptively choose operators based on their past performance. This adaptive strategy allows the algorithm to exploit the strengths of different operators in different states/situations during the search process, thereby improving the effectiveness and efficiency of the search algorithm. By learning which operators are most effective in a given context, AOS can avoid redundant searches, enhance the efficiency of the search process, and ultimately lead to better solutions.

In recent years, learning-based methods [25–30], particularly reinforcement learning (RL), have been increasingly employed in AOS to guide the dynamic selection of operators, significantly enhancing the algorithm's adaptability and performance. By leveraging past performance data, RL-based methods can optimize the search process and improve the overall efficiency of solving complex optimization problems. Although these methods have shown promising results, they still face some limitations. First, most existing research on AOS has focused primarily on the adaptive selection of improvement operators, while the selection of perturbation operators is often handled randomly. This approach neglects the critical role of perturbation operators in diversifying the search process. When perturbation operators are selected arbitrarily, the algorithm may fail to escape local optima or explore the solution space effectively. As a result, the search may become stuck in sub-optimal regions, reducing the overall performance of the algorithm. Second, although reinforcement learning (RL)-based methods aim to learn strategies for operator selection, in some cases the action selection probabilities are gradually adjusted based on past performance. This means that even if a particular improvement operator performs poorly in a given improvement iteration process, it may still have a high probability to be selected due to the smooth probability adjustment. This can result in the continued selection of ineffective operators, leading to resource wastage and ultimately negatively affecting the algorithm's performance.

To address the limitations mentioned above, we aim to propose a Learning Adaptive Neighborhood search with Dual operator Selection (LANDS) method to solve the CVRP problem. The algorithm incorporates two RL-based controllers to simultaneously optimize the selection of improvement and perturbation operators, along with a novel operator filtering mechanism. Extensive experimental results verify the effectiveness of the proposed algorithm. The key objectives with contributions of this work are as follows:

(1) This paper develops a dual RL-based controller framework to simultaneously guide the selection of improvement and perturbation operators. The algorithm adaptively selects the most suitable operator according to the current search state, effectively guiding and accelerating the optimization process toward achieving a (near) optimal solution. In this framework, an appropriate reward strategy is used to effectively capture the varying importance of operators at different stages of the search process.
(2) A new operator filtering mechanism is proposed to dynamically exclude inefficient operators during the search process, reducing computational overhead and enhancing search performance.
(3) The effectiveness and generalization of our proposed hybrid framework are validated on a wide range of CVRP instances.

The rest of this paper is organized as follows. Section 2 provides the literature review. Section 3 describes the proposed method in detail. Section 4 conduct the experiments and analyzes the results. Finally, the conclusion is drawn in Section 5.

2 LITERATURE REVIEW

In this section, we review the existing algorithms for solving the CVRP problem, especially the progress of the Learn-to-Optimize (L2O) method.

2.1 Existing CVRP Algorithms

Over the decades, various algorithmic approaches have been developed to solve CVRP, which can be broadly categorized into exact methods, evolutionary computation (EC) and meta-heuristic techniques, and hyper-heuristic frameworks.

Exact methods, such as branch-and-bound [31], dynamic programming [32], and column generation [33], aim to find optimal solutions by systematically exploring the entire solution space. While exact methods guarantee optimal solutions, they are computationally expensive and impractical for large-scale instances, making them suitable primarily for small- to medium-sized CVRP problems. EC and meta-heuristic methods, such as Genetic Algorithms (GA) [34], Particle Swarm Optimization (PSO) [35], Ant Colony Optimization (ACO) [36], and adaptive neighborhood search [37], continuously improve solutions based on certain heuristics, which are often inspired by natural processes. These approaches are highly scalable and suitable for solving large-scale CVRP instances but may suffer from premature convergence and require careful parameter tuning, including adjusting the execution order of the heuristics. Hyper-heuristics extend meta-heuristics by generating heuristics [38, 39] or incorporating adaptive selection mechanisms, such as Adaptive Operator Selection (AOS) [26, 30], to improve performance in dynamic and complex problem instances.

2.2 Learning-to-Optimize

Recent advancements in machine learning, especially Learn-to-Optimize (L2O) approaches, have introduced new possibilities to solve VRP by combining learning-based strategies with traditional optimization techniques.

L2O hybrid methods integrate traditional heuristics with learning components to boost efficiency and adaptability. For instance, Neural Guided Local Search (NGLS) [40, 41] leverages machine learning to enhance local search, improving convergence in large-scale VRP instances. Similarly, learning-based AOS is crucial for CVRP, enabling dynamic selection of effective operators during optimization. RL-based AOS frameworks typically involve state representation, reward mechanisms, and policy optimization, with studies [26–28] validating their effectiveness.

However, most research focuses solely on optimizing improvement operators, overlooking perturbation operators. For example,

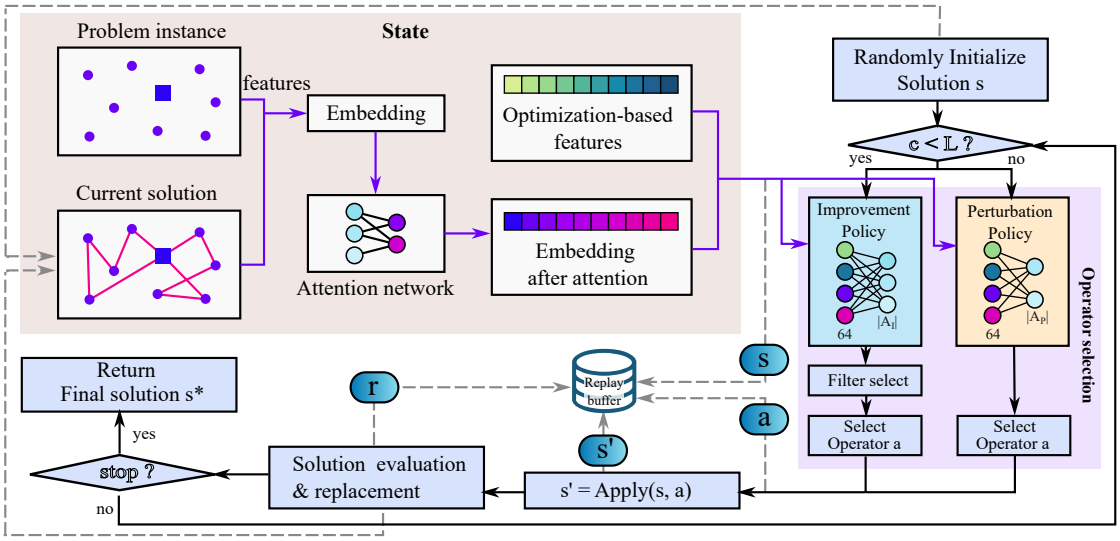


Figure 1: Flowchart of Testing trained LANDS on Test Instances.

Lu et al. [29] proposed the Learn to Improvement (L2I) method to select the improvement operators. These operators are often predefined or randomly selected, lacking performance-based adaptive learning, which hinders exploration of large-scale dynamic problems. Additionally, assigning nonzero probabilities to underperforming operators wastes resources and slows convergence.

To address these issues, we propose a dual-policy learning framework for AOS, optimizing both improvement and perturbation operator selection. A filter mechanism is also introduced to suppress ineffective operators, ensuring resources are allocated to high-performing ones.

3 THE PROPOSED APPROACH

This section describes the newly proposed Learning Adaptive Neighborhood Search with Dual operator Selection (LANDS). In this section, the overall framework is presented first, followed by the description of each component of the algorithm.

3.1 Overall Framework

To address the challenges of solving CVRP and optimize both the improvement and perturbation operator selection, we propose a learning adaptive neighborhood search algorithm integrating two reinforcement learning (RL)-based controllers. These controllers are designed to intelligently guide the selection of operators, ensuring an efficient search process that balances exploration and exploitation. The two controller networks for LANDS share the same architecture: a fully connected network with two layers. The first layer consists of 64 units with a ReLU activation function, while the second layer uses a softmax activation function to generate action probabilities for $|A|$ actions. Fig. 1 illustrates the flowchart of testing the proposed LANDS on the test instance, while Algorithm 1 provides the detailed training steps for dual policies.

The learning process operates in four main stages: initialization, improvement phase, perturbation phase and policy update. During initialization, the improvement controller $\pi_{\theta_I}(\cdot)$ and perturbation controller $\pi_{\theta_P}(\cdot)$ are randomly initiated (line 1). For each episode k, the k-th problem instance is loaded, and the initial solution s is constructed (line 4). Counters for no improvement c, perturbation times np, and improvement times ni are initialized alongside filtered operator set F and experience memory buffer $\mathcal{B}_I$ and $\mathcal{B}_P$ (lines 5-6).

In the adaptive optimization stage, for a problem instance and the current controllers, the solution is iteratively updated T times following the controllers (lines 7-26). The algorithm alternates between the Improvement Phase and the Perturbation Phase based on a threshold-based rule: if no cost reduction is achieved after L consecutive improvement steps, the framework applies a perturbation operator to escape from local optima and restarts the improvement process. During the improvement phase, the framework incorporates an operator filtering mechanism (Detailed in Algorithm 2, which uses the ϵ-greedy strategy) to exclude ineffective operators and select appropriate operator based on improvement controller $\pi_{\theta_I}(\cdot)$ (line 11), then the selected operator is applied, and the resulting best neighbor solution s' is evaluated (line 12). If improves upon s, the solution is updated, and the improvement counter c is reset. Otherwise, the counter c is incremented, and the operator is added to the filtered set F to avoid wasting resources (line 19). When $c \geq L$, the perturbation controller $\pi_{\theta_P}(\cdot)$ is activated to select and apply a perturbation operator, generating a new solution s. The perturbation counter np is incremented, and c and F are reset to restart the improvement process (lines 21-26).

During both phases, the experiences, including state features, operators, and corresponding rewards, are stored in the respective memory buffers $\mathcal{B}_I$ and $\mathcal{B}_P$. After each episode, the controllers $\pi_{\theta_I}(\cdot)$ and $\pi_{\theta_P}(\cdot)$ are updated using randomly sampled mini-batches of experiences from the memory buffers (line 28). The algorithm iteratively refines the solution, leveraging the dual-policy learning mechanism, until the maximum number of episodes NoE is reached. Finally, two trained policies $\pi_{\theta_I}(\cdot)$ and $\pi_{\theta_P}(\cdot)$ are obtained.

Algorithm 1: Dual Policy Learning Process

Input : Maximum episodes NoE, maximum timesteps T, threshold L
Output : the learned policies $\pi_{\theta_I}(\cdot)$ and $\pi_{\theta_P}(\cdot)$.

Randomly initialize the improvement controller $\pi_{\theta_I}(\cdot)$ and perturbation controller $\pi_{\theta_P}(\cdot)$
for *episode* $k = 1$ *to* NoE **do**
  // Initialization
  Load k-th instance and construct initial solution s
  Set no improvement counter $c \leftarrow 0$, perturbation times $np \leftarrow 0$, improvement times $ni \leftarrow 0$, Filtered operator set $F \leftarrow \emptyset$
  Initialize experience memory buffer $\mathcal{B}_I \leftarrow \emptyset$, $\mathcal{B}_P \leftarrow \emptyset$
  for *timestep* $t = 1$ *to* T **do**
    // Improvement Phase
    if $c < L$ **then**
      Construct state features $\phi(s)$ for improvement based on Table 1
      Select available improvement operator: $a_{ni} \leftarrow$ Filter Select$(F, \pi_{\theta_I}, \phi(s))$
      Apply Improvement: $s' \leftarrow$ **Local Search**(s, a_{ni})
      Save experience $\langle \phi(s), a_{ni}, r_{ni}, \phi(s') \rangle$ to $\mathcal{B}_I$
      $ni \leftarrow ni + 1$
      if $Cost(s') < Cost(s)$ **then**
        Update $s \leftarrow s'$ $s^* \leftarrow s$
        Reset $c \leftarrow 0$, $F \leftarrow \emptyset$
      else
        $c \leftarrow c + 1$, $F \leftarrow F \cup \{a_{ni}\}$
    // Perturbation Phase
    else
      $s_{np} \leftarrow s$, construct state $\phi(s_{np})$
      Select: $a_{np} \leftarrow \pi_{\theta_P}(\phi(s_{np}))$
      Apply: $s \leftarrow$ **Perturb**(s_{np}, a_{np})
      Save $\langle \phi(s_{np-1}), a_{np-1}, r_{np-1}, \phi(s_{np}) \rangle$ to $\mathcal{B}_P$
      $np \leftarrow np + 1$, Reset $c \leftarrow 0$, $F \leftarrow \emptyset$
  // Policy learn and update
  Sample random mini-batch of experiences from $\mathcal{B}_I$, $\mathcal{B}_P$ and Update π_{θ_I}, π_{θ_P} using mini-batch respectively
END

During training, the proposed LANDS algorithm is executed for NoE episodes. During testing, it is run for a single episode without applying any updates. This framework efficiently combines local search improvements with perturbation strategies, dynamically adapting the search behavior via learned policies. By integrating dual controllers, it balances exploration and exploitation to enhance performance in complex optimization tasks.

3.2 State Representation

Improvement controller and perturbation controller make decisions based on the input state features, which serve as a comprehensive representation of the optimization environment. Each state comprises features derived from three primary sources: problem instance, solution and optimization process.

Algorithm 2: Filter Select

Input : Filtered operator set F, improvement controller $\pi_{\theta_I}(\phi)$, state $\phi(s)$
Output : Selected operator a

$\mathcal{A}' \leftarrow \mathcal{A}_I \setminus F$
if $random(0, 1) < \epsilon$ **then**
  Randomly select a from $\mathcal{A}'$
else
  Compute and normalize $P_{\mathcal{A}'}$ based on $\pi_{\theta_I}(\phi(s))$
  $a \leftarrow$ RouletteWheelSelection$(\mathcal{A}', P_{\mathcal{A}'})$
return a

Table 1: State Features

Type	Name	Description
Problem-based	c_i	Demand of node i
	(x_i, y_i)	Location of node i
Solution-based	(x_{i^-}, y_{i^-})	Location of node visited before i
	(x_{i^+}, y_{i^+})	Location of node visited after i
	$d_{i^-,i}$	Distance between i^- and i
	$d_{i^+,i}$	Distance between i and i^+
	C_i	the remaining capacity of the vehicle after visiting node i
Optimization	a	Action taken in the previous step
	e	The effect of the previously used action, where $e \in \{-1, 1\}$.
	Δ	Difference between the current solution and the previous solution
	η	Difference between the current solution and the global best solution

The detailed composition of the state features is presented in Table 1. Specifically, problem-based and solution-based features are first transformed into embeddings of a fixed length D, which are then processed by an attention network, configured with 8 heads and D output units, to effectively capture feature interactions. Finally, the output of the attention network is concatenated with optimization-based features, forming the comprehensive state representation used by the controllers, as shown in the upper left corner of Fig. 1. The state representation aligns with the method used in Lu et al. [29]. In this framework, both the perturbation controller and the improvement controller operate within the same state space.

3.3 Action Representation

An action is a behavior chosen by the agent based on its current state. In this framework, an action refers to selecting an improvement operator or a perturbation operator. The detailed descriptions of the improvement and perturbation operators are provided in Table 2 and Table 3, respectively.

Each action in the space is associated with a unique identifier, and the dimensions of the improvement controller and the perturbation controller are determined by the number of operators available for each. The framework dynamically selects actions based on the current state and the decision strategies of the improvement and perturbation controllers using a roulette wheel selection.

Table 2: Descriptions of the Improvement Operators

Type	Name(#operated routes)	Description
Intra-route	2-opt	Reverses a sub-route.
	relocate(1)	Move a segment of m nodes ($m = 1, 2, 3$) in the route to a new location.
	swap(1)	Exchange two nodes in the same route.
Inter-route	cross(2)	Exchange the segments from two routes.
	symmetric-swap(2)	Exchange segments of length m ($m = 1, 2, 3, 4$) between two routes.
	asymmetric-swap(2)	Exchange segments of length m and n ($m = 1, 2, 3, 4, n = 1, 2, 3, 4, m \neq n$) between two routes.
	relocate(2)	Move a segment of length m ($m = 1, 2, 3$) from a route to another.
	cyclic-exchange(3)	Exchange cyclically one customer between three routes.

Table 3: Descriptions of the Perturbation Operators

Name	Description
permute	Randomly destroy m routes and re-construct routes by visiting affected customers in a random order.
cyclic-exchange	Exchange cyclically customers between multiple routes.

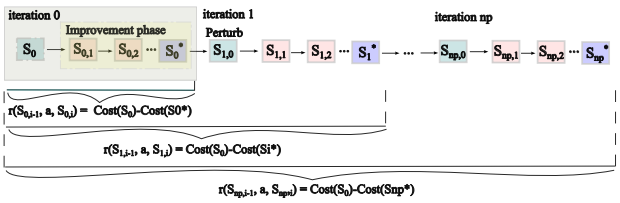


Figure 2: The reward scheme.

3.4 Reward

The reward scheme in LANDS uses an advantage-based mechanism. The initial solution s_0 for the problem instance serves as the baseline for comparison. At each subsequent time step, all operators $j = \{0, 1, 2, \ldots\}$ applied during the same iteration i (comprising one perturbation and the subsequent improvement phase) received a same reward, as illustrated in Fig. 2:

$$r(s_{i,j}, a, s_{i,j+1}) = Cost(s_0) - Cost(s_i^*) \tag{1}$$

This scheme is motivated by the observation that operators often achieve significant distance reductions for newly perturbed solutions, whereas such reductions become increasingly difficult in later steps. This approach ensures fairness by assigning equal rewards to all operators within the same iteration, avoiding bias toward operators improving earlier in the process.

3.5 controller Update

We use the REINFORCE algorithm [42] to update the policy parameters of both improvement controller π_{θ_I} and perturbation controller π_{θ_P} by optimizing the expected return of the policy. The controller gradient update is computed using the following formula:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} r_t \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \tag{2}$$

where $\nabla_\theta \log \pi_\theta(a_t|s_t)$ is the gradient of the action probability with respect to θ.

The policy parameters are updated by:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta) \tag{3}$$

where α is the learning rate. In each episode, gradients are estimated from sampled trajectories, and the controller is updated to improve the action selection, leading to better overall performance.

In this work, an episode is defined as the whole optimization process of a problem instance. For training purposes, the proposed LANDS is run for NoE episodes. For testing purposes, the proposed LANDS is executed for one episode without any updates.

4 EXPERIMENTS

In this section, we construct a series of CVRP instances following the CVRP instance generation protocol established by Uchoa et al. [43] to thoroughly evaluate the performance of the proposed algorithm. The experimental results are carefully analyzed in detail.

4.1 Datasets

To evaluate the proposed algorithm, this study focuses on three CVRP datasets, each characterized by different numbers of customers: $N = 20, 50$, and 100. The spatial distribution of both customer locations and the depot (x_i, y_i) is uniformly random within the interval $[0, 1]$. Each customer's demand c_i is randomly sampled from the discrete set $\{1, 2, \ldots, 9\}$, simulating varying levels of resource requirements across the network. When $N = 20, 50, 100$, the vehicle capacities are set to 20, 40, and 50, respectively.

4.2 Experiment Settings

To train the policy network, we used the ADAM optimizer with a learning rate of 0.001, and the maximum number of iterations was set to $L = 6$, in line with [29]. Each dataset was trained using 500 problem instances ($NoE = 500$). For each problem instance and the given controllers, the solution was iteratively updated for $T = 20,000$ times. The trained models were tested on 500 unseen instances, and the average distance was calculated as the objective value:

$$\min\ f = \frac{1}{N} \sum_{i=1}^{N} d(I_i) \tag{4}$$

where $d(I_i)$ represents the total distance of the solution for the i-th test instance.

All experiments were independently run 30 times on the test set, running on a Linux server. The hardware configuration of the server consists of dual AMD EPYC 7702 64-Core Processors (256 threads) with 502GB of RAM.

Table 4: Computational Results (Average Total Distance over 500 Test Instances) Obtained by Different Algorithm

		CVRP20	CVRP50	CVRP100
LKH3 [44]		6.0863	10.3499	15.6716
ACO [45]	best	6.3097	11.3401	18.3076
	mean	6.3121 (↑)	11.3527 (↑)	18.3213 (↑)
	std	0.0015	0.0053	0.0064
LEHD [46]	best	6.0904	10.4771	15.8419
	mean	6.0915 (↑)	10.4856 (↑)	15.8514 (↑)
	std	0.0006	0.0045	0.0065
L2I [29]	best	6.081	10.3607	15.6663
	mean	6.082 (↑)	10.3787 (↑)	15.7334 (↑)
	std	0.0008	0.0137	0.0524
LANDS_IR	best	6.103	10.5121	15.9871
	mean	6.1322 (↑)	10.6136 ↑	16.2212 (↑)
	std	0.0137	0.0467	0.0908
LANDS_FR	best	6.0817	10.3787	15.7278
	mean	6.0838 (↑)	10.3963 (↑)	15.7959 (↑)
	std	0.001	0.0091	0.05
LANDS_WF	best	6.0807	10.3436	15.6065
	mean	6.0812 (↑)	10.3551 (↑)	15.653 (↑)
	std	0.0004	0.006	0.024
LANDS	best	**6.0805**	**10.3345**	**15.5976**
	mean	**6.0808**	**10.3408**	**15.6381**
	std	0.0002	0.004	0.0198

4.3 Comparative Algorithms

To evaluate the performance of LANDS on the CVRP, we conducted a comparative study against several benchmark algorithms. These include the classic state-of-the-art heuristic algorithm LKH3 [44], an EC algorithm Ant Colony Optimization (ACO) [45], and two learning-based methods, Light Encoder and Heavy Decoder (LEHD) [46] and Learn to Improve (L2I) [29]. Among them, L2I only uses the RL-based improvement controller while randomly selecting the perturbation operator. The LEHD model can learn to dynamically capture the relationship between all available nodes, which vary in size. The number of tasks in the training set in LEHD is 100, which is consistent with the original paper. For fair comparison, in LEHD, the model is trained using a dataset consisting of 10,000 problem instances (ensuring that the training time is not much different from the proposed method). For all comparison algorithms, the parameter settings are applied as recommended in their respective papers to ensure a fair evaluation.

In addition to these benchmark algorithms, our aim is also to validate the effectiveness of our filter scheme and reward strategy. For this purpose, we introduce three additional variants of the LANDS algorithm: (1) LANDS_WF, which does not incorporate the filter mechanism. (2) LANDS_IR, which uses an immediate reward scheme, where $r(s_{i,j}, a, s_{i,j+1}) = Cost(s_{i,j}) - Cost(s_{i,j+1})$. (3) LANDS_FR, which uses a reward structure that all operators applied in the same iteration received the same reward but does not address diminishing returns: $r(s_{i,j}, a, s_{i,j+1}) = Cost(s^*_{i-1}) - Cost(s^*_i)$.

4.4 Results and Discussions

The result is average total distance over 500 test instances, which is calculated as Eq.(4), and the value is the smaller the better. We calculated the minimum, mean, and variance of each algorithm when it was run 30 times independently on each dataset. The comparison results are presented in Table 4, where each row represents an algorithm, and each column corresponds to a CVRP dataset. The best minimum and mean values are highlighted. Statistical significance is assessed using the Wilcoxon rank-sum test at a significance level of 0.05, which '↑', '↓' and '≈' denote that the algorithm is significantly worse than, better than, and is equal to LANDS, respectively.

LANDS achieves the best results in all three problem sizes with the smallest standard deviation, indicating the most stable performance among all algorithms tested. ACO and LEHD exhibit competitive performance in smaller instances, but still have a gap with LANDS on larger instances. L2I, which employs an RL-based improvement controller with random perturbation operator selection, yields improved results over LKH3 but is outperformed by LANDS, emphasizing the limitation of random perturbation operator selection, especially for larger-scale problems. Compared with LANDS_WF, the variant without the filter scheme performs slightly worse than LANDS but still outperforms LKH3 and L2I, highlighting the benefit of the filter scheme in improving solution quality. Comparisons with LANDS_IR and LANDS_FR underscore the importance of accounting for diminishing returns in the reward structure.

Moreover, the convergence curves for each CVRP dataset, shown in Figure 3, reveal that as problem size increases, the performance gap between LANDS and other AOS-based algorithms becomes more pronounced.

In conclusion, the results validate the necessity of the dual policy learning mechanisms in LANDS. By leveraging both the filter scheme and an appropriate reward structure, LANDS achieves better performance across all problem sizes, outperforming benchmark algorithms and its own variants.

Table 5: Computational Results (Average Total Distance over 500 Test Instances) of Different Perturbation Strategies

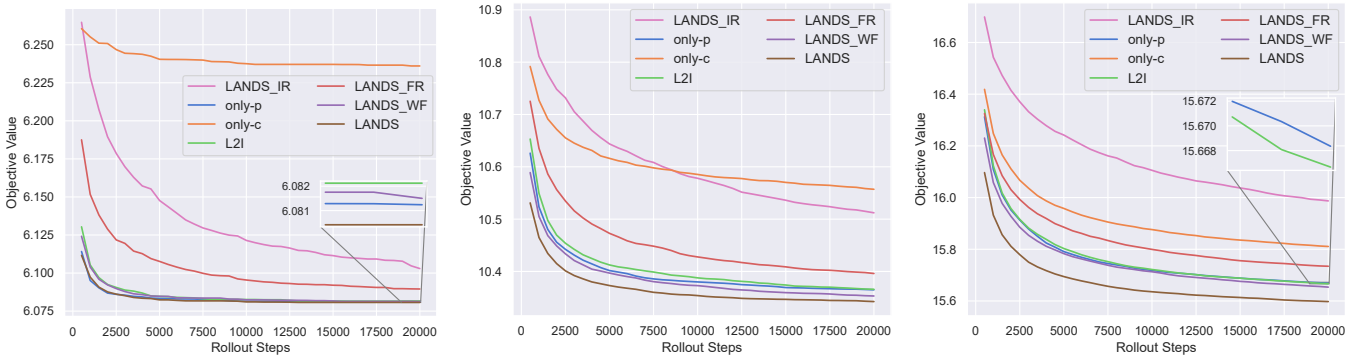
		CVRP20	CVRP50	CVRP100
Only-p	best	**6.0806**	**10.3591**	**15.6656**
	mean	**6.0815**	**10.3776**	15.7593
	std	0.001	0.0152	0.0747
Only-c	best	6.2262	10.557	15.8033
	mean	6.242	10.5732	15.8297
	std	0.0069	0.011	0.0177
L2I	best	6.081	10.3607	15.6663
	mean	6.082	10.3787	**15.7334**
	std	0.0008	0.0137	0.0524

4.5 Necessity of Perturbation Controller

To verify the necessity of learning an adaptive perturbation controller within the RL-based optimizer, we first compare three strategies: (1) Only-p, using only the permute operator; (2) Only-c, using only the cyclic-exchange operator; and (3) L2I, randomly selecting between the two operators.

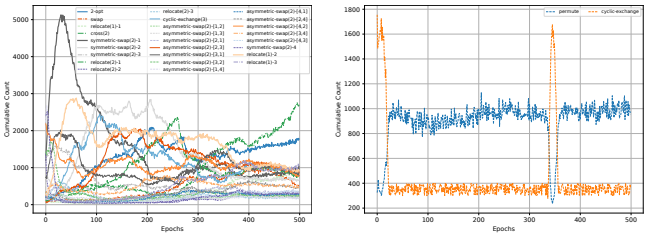
The results from the experiment, as shown in Table 5, reveal an interesting observation. Although Only-p achieves the best results on small-scale instances, it underperforms when applied to larger problem instances. In contrast, L2I, which alternates between the two operators, consistently achieves comparable or superior results across all problem sizes. This suggests that operator cyclic-exchange can provide essential improvements in some special cases, which highlights the necessity of using both operators effectively.

These findings highlight the need for an adaptive strategy to dynamically select perturbation operators based on the current



(a) $N = 20$ (b) $N = 50$ (c) $N = 100$

Figure 3: Convergence curves of LANDS and different AOS-based methods.



(a) Improvement Operators (b) Perturbation Operators

Figure 4: Pattern of operator usages as CVRP 100 training epoch grows.

state of the optimization process. By leveraging both operators effectively, the algorithm can maintain robust performance across varying problem complexities.

4.6 Analysis of Operator Usages

Fig. 4 illustrates the usage frequency of both the improvement and perturbation operators over 500 episodes during the CVRP100 optimization process. Fig. 4(a) highlights the cumulative usage of various improvement operators, while Fig. 4(b) focuses on the cumulative usage of perturbation operators.

In Fig. 4(a), the selection frequency of each operator exhibits dynamic changes throughout the optimization process. The operators are distinguished by "name(#operated routes)-x", where "x" represents the length of the segment m. During the initial training phase, the algorithm favors operators with larger-scale modifications, such as "symmetric-swap(2)-1," "symmetric-swap(2)-2," and "cyclic-exchange(3)," to facilitate broader exploration of the solution space. However, as training progresses, operators with finer-grained adjustments, like "2-opt," dominate in cumulative usage, highlighting their efficiency during local improvement phases. In contrast, operators such as "asymmetric-swap(2)-[1,4]" and "relocate(2)-3" have the lowest usage frequencies, indicating their limited applicability to specific problem states.

Fig. 4(b) demonstrates a dynamic adjustment in the usage of perturbation operators. Initially, "cyclic-exchange" is heavily used, likely to explore diverse solutions early in the optimization process. However, as the process progresses, the usage of "permute" dominates, indicating its suitability for fine-tuning solutions in later stages. A noticeable spike in "cyclic-exchange" usage is observed around the 300th epoch, possibly triggered by the algorithm encountering challenging local optima requiring more aggressive exploration. Further analysis of the instances shows that these scenarios are characterized by fragmented customer distributions and increased route imbalances, which create a more complex optimization landscape. This complexity likely traps the algorithm in challenging local optima, requiring the aggressive exploration provided by "cyclic-exchange" to identify better solutions.

Overall, the figures reveal the adaptive nature of operator selection within the AOS framework. The algorithm effectively balances the use of operators based on the optimization stage and problem state.

Table 6: Generalization Performance (Average Total Distance over 500 Test Instances) on CVRP Instances of Different Scales

		20 ⟶ 50	20 ⟶ 100	50 ⟶ 100
L2I	best	10.3506	15.6293	15.6328
	mean	10.3628 (↑)	15.7029 (↑)	15.6714 (↑)
	std	0.0087	0.0809	0.0456
LANDS_FR	best	10.3864	15.7151	15.6753
	mean	10.4075 (↑)	15.8201 (↑)	15.7161 (↑)
	std	0.0139	0.062	0.0286
LANDS_WF	best	10.3514	15.6383	15.5915
	mean	10.3607 (↑)	15.7416 (↑)	15.6303 (↑)
	std	0.006	0.0456	0.0319
LANDS	best	**10.3335**	**15.5722**	**15.5742**
	mean	**10.3409**	**15.6039**	**15.5926**
	std	0.0035	0.0156	0.0121

4.7 Generalization and Adaptability Analysis

This section examines the ability of the proposed algorithm to generalize across different problem scales and its adaptability to diverse benchmark datasets.

Table 6 demonstrates the transferability of algorithms trained in smaller scale instances to larger scale test problems (e.g. 20 ⟶ 50 means the model trained in CVRP20 tests on CVRP50). LANDS

Table 7: Performance Comparison on CVRP Library Benchmarks

		BKS	LEHD [46]	L2I [29]	LANDS_FR	LANDS_WF	LANDS
X-n101-k25	best	27591	28145.61	28076.85	28105.19	27949.44	27615.38
	mean		28386.42	28417.64	28665.61	28215.93	27927.75
	std		130.69	179.16	364.26	230.87	136.18
X-n261-k13	best	26558	27514.34	28608.15	27978.60	27870.97	27328.93
	mean		28013.16	29250.14	29168.33	28517.15	27688.22
	std		258.51	275.68	411.38	292.68	182.70
X-n331-k15	best	31102	32336.99	33359.79	33149.15	32647.57	31739.20
	mean		33020.61	33839	33742.30	33086.90	32044.11
	std		299.57	307.24	286.12	293.92	162.24
X-n420-k130	best	107798	119403.97	110092.01	110033.60	109654.01	109056.53
	mean		120437.96	111104.40	110513.50	110463.63	109522.05
	std		628.52	325.61	272.62	338.69	219.12
X-n513-k21	best	24201	27052.21	26462.71	25883.72	25542.67	24841.78
	mean		27532.85	27273.49	26502.55	26274.82	25005.01
	std		250.29	501.66	342.97	321.21	92.18
X-n613-k62	best	59535	65013.57	64333.62	63081.26	62680.16	61516.42
	mean		65701.16	65365.06	63993.24	63454.80	61819.20
	std		286.52	591.02	494.97	406.13	164.04
X-n701-k44	best	81923	88488.54	92034.43	87296.25	86758.14	84917.64
	mean		89316.59	94453.01	88103.95	87580.22	85475.93
	std		421.9	1056.73	491.74	472.02	230.65
X-n801-k40	best	73311	79154.95	88188.46	76627.53	76845.34	75260.26
	mean		80290.66	93258.41	77648.46	77688.38	75932.65
	std		442.79	5127.52	473.95	454.83	218.41
X-n916-k207	best	329179	354583.06	363036.36	335865.11	336318.57	335468.51
	mean		357081.81	368784.99	337123.71	337160.32	336456.27
	std		946.27	2383.99	545.28	480.13	394.53
X-n1001-k43	best	72355	80981.09	92196.18	78625.57	78691.31	76457.87
	mean		82048.95	96627.31	80205.35	79453.68	77116.93
	std		551.22	1961.46	627.35	585.54	285.45

achieves the best results across all test cases, indicating superior generalization and robustness. Notably, LANDS consistently outperforms other variants, such as LANDS_FR and LANDS_WF, and the baseline L2I, especially in larger problem instances like 50 $\rightarrow$ 100, where it achieves a significant improvement in both solution quality and stability.

Table 7 evaluates the performance (the value of total distances) of different algorithms on CVRP library benchmarks. The models are all trained on CVRP50, except LEHD. Although it does not reach the best solution (BKS), LANDS shows consistently better solution quality compared to LEHD, L2I and other LANDS variants. These results verify the generalization ability of LANDS.

5 CONCLUSIONS

In this paper, we develop a learning adaptive neighborhood search with dual operator selection (LANDS) algorithm for Capacitated Vehicle Routing Problems. By integrating two reinforcement learning-based controllers, LANDS adaptively selects both improvement and perturbation operators, enhancing the exploration and exploitation balance during the search process. Additionally, the introduction of a filtering mechanism further improves efficiency by eliminating redundant operator evaluations. Extensive experimental results on various datasets show that LANDS outperforms the comparison algorithms in terms of solution quality and generalization.

A significant challenge in perturbation-based optimization lies in accurately evaluating the reward associated with perturbation operators, given their inherent randomness. Existing approaches often fail to fully capture the stochastic nature of perturbations, which can lead to suboptimal operator selection. Future work should focus on developing more robust reward evaluation methods that better reflect the impact of perturbation actions. This could involve adapting the reward structure to consider the variability and success rate of perturbations, enabling more informed decisions and improving overall algorithm performance in dynamic problem settings.

REFERENCES

[1] Asma M Altabeeb, Abdulqader M Mohsen, Laith Abualigah, and Abdullatif Ghallab. Solving capacitated vehicle routing problem using cooperative firefly algorithm. *Applied Soft Computing*, 108:107403, 2021.

[2] James Fitzpatrick, Deepak Ajwani, and Paula Carroll. A scalable learning approach for the capacitated vehicle routing problem. *Computers & Operations Research*, 171:106787, 2024.

[3] George B Dantzig and John H Ramser. The truck dispatching problem. *Management science*, 6(1):80–91, 1959.

[4] Neda Tanoumand and Tonguç Ünlüyurt. An exact algorithm for the resource constrained home health care vehicle routing problem. *Annals of Operations Research*, 304:397–425, 2021.

[5] Gilbert Laporte and Yves Nobert. Exact algorithms for the vehicle routing problem. In *North-Holland mathematics studies*, volume 132, pages 147–184. Elsevier, 1987.

[6] Maria Battarra, Güneş Erdoğan, and Daniele Vigo. Exact algorithms for the clustered vehicle routing problem. *Operations Research*, 62(1):58–71, 2014.

[7] David Pisinger and Stefan Ropke. A general heuristic for vehicle routing problems. *Computers & operations research*, 34(8):2403–2435, 2007.

[8] Túlio AM Toffolo, Thibaut Vidal, and Tony Wauters. Heuristics for vehicle routing problems: Sequence or set optimization? *Computers & operations research*, 105:118–131, 2019.

[9] Xiaoyun Xia, Helin Zhuang, Zijia Wang, and Zefeng Chen. Two-stage heuristic algorithm with pseudo node-based model for electric vehicle routing problem. *Applied Soft Computing*, 165:112102, 2024.

[10] Ya-Hui Jia, Yi Mei, and Mengjie Zhang. Confidence-based ant colony optimization for capacitated electric vehicle routing problem with comparison of different encoding schemes. *IEEE Transactions on Evolutionary Computation*, 26(6):1394–1408, 2022.

[11] Thibaut Vidal. Hybrid genetic search for the cvrp: Open-source implementation and swap* neighborhood. *Computers & Operations Research*, 140:105643, 2022.

[12] Yiğit Çağatay Kuyu and Fahri Vatansever. A hybrid approach of alns with alternative initialization and acceptance mechanisms for capacitated vehicle routing problems. *Cluster Computing*, pages 1–24, 2024.

[13] Xiao-Cheng Liao, Wei-Neng Chen, Xiao-Qi Guo, Jinghui Zhong, and Xiao-Min Hu. Crowd management through optimal layout of fences: An ant colony approach based on crowd simulation. *IEEE Transactions on Intelligent Transportation Systems*, 24(9):9137–9149, 2023.

[14] Xinyu Wang, Tsan-Ming Choi, Zhiying Li, and Shuai Shao. An effective local search algorithm for the multidepot cumulative capacitated vehicle routing problem. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 50(12):4948–4958, 2019.

[15] Ran Liu and Zhibin Jiang. A hybrid large-neighborhood search algorithm for the cumulative capacitated vehicle routing problem with time-window constraints. *Applied Soft Computing*, 80:18–30, 2019.

[16] Chris Groër, Bruce Golden, and Edward Wasil. A library of local search heuristics for the vehicle routing problem. *Mathematical Programming Computation*, 2:79–101, 2010.

[17] MF Ibrahim, FR Nurhakiki, DM Utama, and AA Rizaki. Optimised genetic algorithm crossover and mutation stage for vehicle routing problem pick-up and delivery with time windows. In *IOP Conference Series: Materials Science and Engineering*, volume 1071, page 012025. IOP Publishing, 2021.

[18] Xiao-Cheng Liao, Xiao-Min Hu, Xiang-Ling Chen, Yi Mei, Ya-Hui Jia, and Wei-Neng Chen. Learning to communicate among agents for large-scale dynamic path planning with genetic programming hyper-heuristic. *IEEE Transactions on Artificial Intelligence*, 2024.

[19] Rekik Ali, Gabsi Mounir, and Temani Moncef. Adaptive probabilities of crossover and mutation in genetic algorithm for solving stochastic vehicle routing problem. *International Journal of Advanced Intelligence Paradigms*, 8(3):318–326, 2016.

[20] Jiyuan Pei, Yi Mei, Jialin Liu, and Xin Yao. An investigation of adaptive operator selection in solving complex vehicle routing problem. In *Pacific Rim International Conference on Artificial Intelligence*, pages 562–573. Springer, 2022.

[21] Xiang-Ling Chen, Xiao-Cheng Liao, Feng-Feng Wei, and Wei-Neng Chen. An order-aware adaptive iterative local search metaheuristic for multi-depot uav pickup and delivery problem. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 1183–1191, 2024.

[22] Jiyuan Pei, Hao Tong, Jialin Liu, Yi Mei, and Xin Yao. Local optima correlation assisted adaptive operator selection. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 339–347, 2023.

[23] Richard J Marshall, Mark Johnston, and Mengjie Zhang. Hyper-heuristic operator selection and acceptance criteria. In *Evolutionary Computation in Combinatorial Optimization: 15th European Conference, EvoCOP 2015, Copenhagen, Denmark, April 8-10, 2015, Proceedings 15*, pages 99–113. Springer, 2015.

[24] Cheng Chen, Emrah Demir, and Yuan Huang. An adaptive large neighborhood search heuristic for the vehicle routing problem with time windows and delivery robots. *European journal of operational research*, 294(3):1164–1180, 2021.

[25] Ye Tian, Xiaopeng Li, Haiping Ma, Xingyi Zhang, Kay Chen Tan, and Yaochu Jin. Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 7(4):1051–1064, 2022.

[26] Rafet Durgut, Mehmet Emin Aydin, and Ibrahim Atli. Adaptive operator selection with reinforcement learning. *Information Sciences*, 581:773–790, 2021.

[27] Rafet Durgut, Mehmet Emin Aydin, and Abdur Rakib. Transfer learning for operator selection: A reinforcement learning approach. *Algorithms*, 15(1):24, 2022.

[28] Stephanus Daniel Handoko, Duc Thien Nguyen, Zhi Yuan, and Hoong Chuin Lau. Reinforcement learning for adaptive operator selection in memetic search applied to quadratic assignment problem. In *Proceedings of the companion publication of the 2014 annual conference on genetic and evolutionary computation*, pages 193–194, 2014.

[29] Hao Lu, Xingwen Zhang, and Shuang Yang. A learning-based iterative method for solving vehicle routing problems. In *International conference on learning representations*, 2019.

[30] Wenjie Yi, Rong Qu, Licheng Jiao, and Ben Niu. Automated design of metaheuristics using reinforcement learning within a novel general search framework. *IEEE Transactions on Evolutionary Computation*, 27(4):1072–1084, 2022.

[31] Claudia Archetti, Nicola Bianchessi, and M Grazia Speranza. Branch-and-cut algorithms for the split delivery vehicle routing problem. *European Journal of Operational Research*, 238(3):685–698, 2014.

[32] Wei Ou and Bao-Gang Sun. A dynamic programming algorithm for vehicle routing problems. In *2010 International Conference on Computational and Information Sciences*, pages 733–736. IEEE, 2010.

[33] Martin Fink, Guy Desaulniers, Markus Frey, Ferdinand Kiermaier, Rainer Kolisch, and François Soumis. Column generation for vehicle routing problems with multiple synchronization constraints. *European Journal of Operational Research*, 272(2):699–711, 2019.

[34] Abhay Sobhanan, Junyoung Park, Jinkyoo Park, and Changhyun Kwon. Genetic algorithms with neural cost predictor for solving hierarchical vehicle routing problems. *Transportation Science*, 2024.

[35] Qichao Wu, Xuewen Xia, Haojie Song, Hui Zeng, Xing Xu, Yinglong Zhang, Fei Yu, and Hongrun Wu. A neighborhood comprehensive learning particle swarm optimization for the vehicle routing problem with time windows. *Swarm and Evolutionary Computation*, 84:101425, 2024.

[36] Zhaojun Zhang, Simeng Tan, Jiale Qin, Kuansheng Zou, and Shengwu Zhou. Multi-strategy ant colony optimization with k-means clustering algorithm for capacitated vehicle routing problem. *Cluster Computing*, 28(3):1–24, 2025.

[37] Stefan Voigt. A review and ranking of operators in adaptive large neighborhood search for vehicle routing problems. *European Journal of Operational Research*, 2024.

[38] Xiao-Cheng Liao, Yi Mei, and Mengjie Zhang. Learning traffic signal control via genetic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 924–932, 2024.

[39] Jaromír Mlejnek and Jiří Kubalik. Evolutionary hyperheuristic for capacitated vehicle routing problem. In *Proceedings of the 15th annual conference companion on Genetic and evolutionary computation*, pages 219–220, 2013.

[40] Benjamin Hudson, Qingbiao Li, Matthew Malencia, and Amanda Prorok. Graph neural network guided local search for the traveling salesperson problem. *arXiv preprint arXiv:2110.05291*, 2021.

[41] Nasrin Sultana, Jeffrey Chan, Babak Abbasi, Tabinda Sarwar, and AK Qin. Learning to guide local search optimisation for routing problems. *Operations Research Letters*, page 107136, 2024.

[42] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.

[43] Eduardo Uchoa, Diego Pecin, Artur Pessoa, Marcus Poggi, Thibaut Vidal, and Anand Subramanian. New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 257(3):845–858, 2017.

[44] Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde: Roskilde University*, 12:966–980, 2017.

[45] Mélanie Suppan, Thomas Hanne, and Rolf Dornberger. Ant colony optimization to solve the rescue problem as a vehicle routing problem with hard time windows. In *Proceedings of International Joint Conference on Advances in Computational Intelligence: IJCACI 2021*, pages 53–65. Springer, 2022.

[46] Fu Luo, Xi Lin, Fei Liu, Qingfu Zhang, and Zhenkun Wang. Neural combinatorial optimization with heavy decoder: Toward large scale generalization. *Advances in Neural Information Processing Systems*, 36:8845–8864, 2023.